



درس برنامه نویسی همروند
نیم سال دوم ۱۴۰۴-۱۴۰۵
استاد: مهدی موحیدیان مقدم
دستیار آموزشی: سبا سلطانی فیروز

تمرین سری سوم

- پرسش‌های خود را می‌توانید در گروه متصل به کانال تلگرامی درس (@concurrent4042) و در صورت عدم دسترسی به اینترنت بین‌الملل در گروه بله^۱، مطرح کنید.
- پاسخ‌های خود را در سامانه LMS ارسال فرمایید.
- نوشتن مشخصات خود را فراموش نفرمایید.

۱ مقدمه

در بسیاری از مسائل علمی و مهندسی، محاسبه انتگرال معین یکی از عملیات‌های پایه و پرتکرار است. در بسیاری از کاربردها، مقدار انتگرال به صورت تحلیلی قابل محاسبه نیست یا محاسبه دقیق آن دشوار و زمان‌بر است. در چنین شرایطی، از روش‌های عددی برای تقریب مقدار انتگرال استفاده می‌شود. یکی از ساده‌ترین و شناخته‌شده‌ترین روش‌های عددی برای تقریب انتگرال، قاعده دوزنقه‌ای است. در این روش، بازه انتگرال به تعدادی زیربازه کوچک‌تر تقسیم شده و مساحت زیر نمودار تابع با مجموع مساحت تعدادی دوزنقه تقریب زده می‌شود. از آنجا که محاسبه سهم هر زیربازه تا حد زیادی مستقل از سایر زیربازه‌ها است، این روش به صورت طبیعی قابلیت موازی‌سازی دارد. هدف این پروژه، پیاده‌سازی سریال و موازی قاعده دوزنقه‌ای و تحلیل صحت و کارایی پیاده‌سازی موازی است. در این پروژه، علاوه بر محاسبه مقدار عددی انتگرال، لازم است مفاهیمی مانند تقسیم کار، دسترسی همزمان به داده مشترک، شرایط رقابتی، همگام‌سازی، زمان اجرا، افزایش سرعت و کارایی موازی مورد بررسی قرار گیرند.

۲ شرح مسئله

انتگرال معین زیر را در نظر بگیرید:

$$I = \int_a^b f(x) dx \quad (1)$$

هدف، تقریب مقدار این انتگرال با استفاده از قاعده دوزنقه‌ای است. در قاعده دوزنقه‌ای، بازه $[a, b]$ به n زیربازه مساوی تقسیم می‌شود. اگر طول هر زیربازه برابر با

$$h = \frac{b - a}{n} \quad (2)$$

¹<https://ble.ir/join/GriwxnAhmG>

باشد و نقاط شبکه به صورت

$$x_i = a + ih, \quad i = 0, 1, \dots, n \quad (۳)$$

تعریف شوند، آنگاه تقریب ذوزنقه‌ای انتگرال به صورت زیر محاسبه می‌شود:

$$I \approx h \left[\frac{f(x_0)}{2} + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + \frac{f(x_n)}{2} \right] \quad (۴)$$

در این پروژه باید مقدار انتگرال برای یک تابع مشخص محاسبه شود. به عنوان نمونه، می‌توانید از تابع زیر استفاده کنید:

$$f(x) = \sin(x) \quad (۵)$$

و مقدار انتگرال را روی بازه $[0, \pi]$ محاسبه نمایید:

$$I = \int_0^{\pi} \sin(x) dx \quad (۶)$$

جواب دقیق این انتگرال برابر است با:

$$I_{\text{exact}} = 2 \quad (۷)$$

دانشجو می‌تواند در گزارش خود علاوه بر این تابع، تابع دیگری را نیز به دلخواه بررسی کند؛ اما بررسی تابع بالا الزامی است.

۳ روش حل سریال

در مرحله اول، باید برنامه‌ای سریال برای محاسبه انتگرال با قاعده ذوزنقه‌ای پیاده‌سازی شود. در نسخه سریال، تمام زیربازه‌ها توسط یک جریان اجرای واحد محاسبه می‌شوند و مجموع نهایی به صورت ترتیبی به دست می‌آید. هدف از این مرحله، ایجاد یک مبنای مقایسه برای نسخه موازی است. زمان اجرای نسخه سریال باید اندازه‌گیری شود تا در مراحل بعد بتوان معیارهایی مانند speedup و efficiency را محاسبه کرد.

۴ روش حل موازی

در مرحله دوم، باید نسخه موازی برنامه پیاده‌سازی شود. ایده اصلی در موازی‌سازی آن است که زیربازه‌های انتگرال بین چند Thread تقسیم شوند. هر Worker بخشی از بازه را محاسبه کرده و یک مجموع محلی تولید می‌کند. در پایان، مجموع‌های محلی با یکدیگر ترکیب شده و مقدار نهایی انتگرال به دست می‌آید. در این پروژه، منظور از Worker یک واحد اجرایی مبتنی بر Thread است. بنابراین، استفاده از روش‌های مبتنی بر چندپردازش‌های مانند multiprocessing، ProcessPoolExecutor یا MPI مجاز نیست. هدف پروژه آن است که دانشجو با مفاهیم برنامه‌نویسی مبتنی بر نخ، حافظه مشترک، دسترسی همزمان، ناحیه بحرانی و همگام‌سازی درگیر شود. پیاده‌سازی موازی می‌تواند با یکی از روش‌های زیر انجام شود:

- استفاده از OpenMP در زبان C/C++

● استفاده از threading در زبان Python

● استفاده از ThreadPoolExecutor در زبان Python

دانشجویانی که از OpenMP استفاده می‌کنند، بهتر است از سازوکارهایی مانند parallel for و reduction برای تقسیم حلقه و جمع کردن نتایج استفاده کنند. همچنین، می‌توانند نسخه‌ای با critical نیز پیاده‌سازی کرده و عملکرد آن را با نسخه مبتنی بر reduction مقایسه کنند.

دانشجویانی که از Python threading یا ThreadPoolExecutor استفاده می‌کنند، باید بازه محاسباتی را بین چند Thread تقسیم کنند. هر Thread باید مجموع محلی خود را محاسبه کند و در پایان، نتایج محلی با یک روش مشخص و امن تجمیع شوند. استفاده از مجموع‌های محلی و تجمیع نهایی معمولاً روش مناسب‌تری نسبت به به‌روزرسانی مستقیم یک متغیر مشترک است.

در صورتی که پیاده‌سازی با زبان Python انجام شود، لازم است دانشجو در گزارش خود توضیح دهد که وجود GIL چه اثری بر اجرای موازی محاسبات CPU-bound دارد و چرا ممکن است افزایش تعداد Thread ها الزاماً باعث کاهش زمان اجرا نشود. با این حال، استفاده از threading برای نمایش مفاهیمی مانند تقسیم کار، دسترسی همزمان، race condition و استفاده از Lock قابل قبول است.

۵ بررسی خطای همزمانی

یکی از اهداف مهم این پروژه، درک مشکل دسترسی همزمان به داده مشترک است. بنابراین، لازم است دانشجو علاوه بر نسخه درست برنامه، یک نسخه ناقص یا نادرست نیز بررسی کند. در این نسخه، چند Thread باید تلاش کنند مقدار یک متغیر مشترک را بدون سازوکار مناسب همگام‌سازی به‌روزرسانی کنند.

سپس باید توضیح داده شود که چرا این کار می‌تواند باعث ایجاد race condition شود. همچنین باید نشان داده شود که چگونه می‌توان این مشکل را با استفاده از روش‌هایی مانند critical، reduction، Lock یا جمع محلی و تجمیع نهایی برطرف کرد.

در پیاده‌سازی با OpenMP، استفاده از reduction برای جمع کردن نتایج محلی توصیه می‌شود. همچنین می‌توان یک نسخه با critical پیاده‌سازی کرد و نشان داد که چرا استفاده بیش از حد از ناحیه بحرانی ممکن است باعث کاهش کارایی شود. در پیاده‌سازی با Python threading، می‌توان برای محافظت از متغیر مشترک از Lock استفاده کرد. همچنین می‌توان طراحی بهتری انجام داد که در آن هر Thread مقدار محلی خود را در یک خانه جداگانه ذخیره کند و پس از پایان کار همه Thread ها، جمع نهایی به‌صورت ترتیبی محاسبه شود.

در صورتی که به دلیل ساختار زبان یا ابزار انتخابی، مشاهده خطای عددی به‌صورت مستقیم دشوار باشد، توضیح مفهومی و ارائه یک مثال کوچک از race condition قابل قبول است.

۶ آزمایش‌ها و تحلیل

پس از پیاده‌سازی، لازم است برنامه از دو جنبه بررسی شود: دقت عددی و کارایی موازی. برای بررسی دقت عددی، مقدار تقریبی انتگرال باید برای چند مقدار مختلف n محاسبه شود. به عنوان نمونه، می‌توانید مقادیر زیر را بررسی کنید:

$$n = 10^3, 10^4, 10^5, 10^6 \quad (۸)$$

سپس مقدار تقریبی به‌دست‌آمده باید با جواب دقیق مقایسه شود و خطای مطلق محاسبه گردد:

$$Error = |I_{\text{approx}} - I_{\text{exact}}| \quad (۹)$$

برای بررسی کارایی موازی، برنامه باید با تعداد مختلف Thread اجرا شود. به عنوان نمونه، می‌توانید تعداد Thread ها را برابر با مقادیر زیر در نظر بگیرید:

$$1, 2, 4, 8 \quad (۱۰)$$

در صورتی که سیستم شما تعداد هسته‌های کمتری دارد، می‌توانید تعداد Thread ها را متناسب با سیستم خود انتخاب کنید. برای هر اجرا، زمان اجرای برنامه باید اندازه‌گیری شود. سپس معیار speedup به صورت زیر محاسبه گردد:

$$Speedup(p) = \frac{T_1}{T_p} \quad (۱۱)$$

که در آن T_1 زمان اجرای برنامه با یک Thread و T_p زمان اجرای برنامه با p Thread است. همچنین معیار efficiency به صورت زیر محاسبه می‌شود:

$$Efficiency(p) = \frac{Speedup(p)}{p} \quad (۱۲)$$

در گزارش باید توضیح داده شود که آیا با افزایش تعداد Thread ها، زمان اجرا کاهش یافته است یا خیر. همچنین باید تحلیل شود که چرا در برخی موارد ممکن است افزایش تعداد Thread ها باعث بهبود قابل توجه نشود یا حتی زمان اجرا را افزایش دهد.

در صورتی که برنامه با Python threading پیاده‌سازی شده باشد، مشاهده speedup پایین یا حتی کندتر شدن برنامه لزوماً نشانه غلط بودن پیاده‌سازی نیست. در این حالت، دانشجو باید اثر GIL، سربار ایجاد و مدیریت Thread ها، و نسبت زمان محاسبه به سربار موازی‌سازی را در گزارش تحلیل کند.

۷ تحلیل روش تقسیم کار

در گزارش باید توضیح داده شود که کار بین Thread ها چگونه تقسیم شده است. برای مثال، در روش ساده می‌توان زیربازه‌ها را به بخش‌های تقریباً مساوی تقسیم کرد، به طوری که هر Thread مسئول محاسبه بخشی از بازه باشد. دانشجویانی که از OpenMP استفاده می‌کنند، می‌توانند اثر زمان‌بندی‌های مختلف را نیز بررسی کنند. به عنوان نمونه، مقایسه بین حالت‌های زیر می‌تواند انجام شود:

```
schedule (static)
schedule (static , 1)
schedule (dynamic)
schedule (guided)
```

این بخش برای دانشجویانی که از OpenMP استفاده می‌کنند، امتیاز اضافه دارد. دانشجویانی که از Python استفاده می‌کنند، می‌توانند به جای آن، روش تقسیم بازه بین Thread ها را توضیح داده و اثر تعداد Thread ها بر زمان اجرا را تحلیل کنند. در هر دو حالت، گزارش باید مشخص کند که آیا تقسیم کار به صورت ایستا و از قبل تعیین شده انجام شده است یا در زمان اجرا بین Thread ها توزیع شده است. همچنین باید توضیح داده شود که آیا همه Thread ها تقریباً مقدار یکسانی کار انجام داده‌اند یا خیر.

۸ خروجی‌های مورد انتظار

دانشجو باید کد کامل پروژه را به همراه یک گزارش تحلیلی ارائه دهد. گزارش باید شامل توضیح مسئله، توضیح روش دوزنقه‌ای، توضیح پیاده‌سازی سریال، توضیح پیاده‌سازی موازی مبتنی بر Thread، نتایج عددی، جدول‌های زمان اجرا، محاسبه speedup و efficiency، و تحلیل نتایج باشد.

در گزارش باید حداقل یک جدول برای بررسی دقت عددی و یک جدول برای بررسی کارایی موازی وجود داشته باشد. همچنین رسم نمودار خطا بر حسب تعداد زیربازه‌ها و نمودار زمان اجرا یا speedup بر حسب تعداد Thread ها امتیاز مثبت دارد. همچنین لازم است دانشجو در گزارش خود توضیح دهد که برای جلوگیری از خطای همزمانی از چه روشی استفاده کرده است. برای مثال، در OpenMP می‌توان از reduction یا critical استفاده کرد و در Python threading می‌توان از Lock یا جمع محلی و تجمیع نهایی استفاده نمود.

۹ نکات مربوط به ابزار پیاده‌سازی

در این پروژه، انتخاب ابزار پیاده‌سازی بین OpenMP و روش‌های مبتنی بر Thread در پایتون آزاد است. با این حال، استفاده از ابزارهای مبتنی بر چندپردازه‌ای مجاز نیست. بنابراین، استفاده از multiprocessing، ProcessPoolExecutor و MPI در این پروژه پذیرفته نمی‌شود.

اگر از OpenMP استفاده می‌شود، نحوه کامپایل و اجرای برنامه باید در گزارش ذکر شود. به عنوان نمونه:

```
gcc -fopenmp main.c -o main
./main
```

اگر از Python threading یا ThreadPoolExecutor استفاده می‌شود، نحوه اجرای برنامه، نسخه پایتون و کتابخانه‌های مورد استفاده باید مشخص شوند. همچنین باید توضیح داده شود که چرا در پایتون ممکن است استفاده از Thread ها برای محاسبات سنگین، افزایش سرعت قابل توجهی ایجاد نکند.

۱۰ توضیحات تکمیلی

هدف اصلی این پروژه صرفاً محاسبه یک انتگرال نیست، بلکه هدف اصلی درک عملی مفاهیم برنامه‌نویسی همروند و موازی مبتنی بر Thread است. بنابراین، کیفیت تحلیل، توضیح روش تقسیم کار، بررسی مشکلات همزمانی، و تفسیر نتایج کارایی نقش مهمی در ارزیابی پروژه خواهد داشت.

دقت عددی بالا به تنهایی کافی نیست. برنامه باید از نظر ساختار موازی، نحوه تجمیع نتایج، جلوگیری از خطای همزمانی، و تحلیل زمان اجرا نیز قابل دفاع باشد. همچنین، گزارش باید به گونه‌ای نوشته شود که خواننده بتواند بفهمد برنامه چگونه اجرا شده، چه پارامترهایی استفاده شده، و نتایج به دست آمده چه معنایی دارند.

با آرزوی موفقیت